ROOMBOOKING

Bilan de projet

Analyse critique — Limites, Risques & Pistes d'amélioration

Document	Bilan de projet
Version	2.0
Date	Avril 2026
Auteur	Klaudia Juhasz
Organisation	MediaSchool IRIS Nice — BTS SIO SLAM
Intervenant	Nicolas Choquet

1. Rappel du projet

RoomBooking est une application web de réservation de salles développée pour MediaSchool IRIS Nice dans le cadre de la réalisation professionnelle BTS SIO SLAM. Le projet a été initialement réalisé en autonomie lors d'une semaine intensive (du 23/02/2026 au 27/02/2026), puis le développement de l'application s'est poursuivi par la suite afin d'enrichir ses fonctionnalités et d'améliorer sa qualité globale.

Élément	Détail
Objectif principal	Digitaliser et automatiser la réservation des espaces de travail
Stack technique	Symfony 7, PHP 8.2, MySQL 8, Twig, Bootstrap 5, Docker
Profils utilisateurs	Administrateur, Coordinateur (intervenant), Étudiant
Durée de réalisation	23 jours (du 23 février au 20 mars 2026)
Fonctionnalités livrées	20 tâches couvrant l'intégralité du cahier des charges

2. Bilan des objectifs

L'ensemble des fonctionnalités du cahier des charges a été implémenté et livré dans les délais impartis.

Fonctionnalité	Statut	Remarque
Authentification sécurisée (login/logout/inscription)	Livré	CSRF, bcrypt, rôles Symfony
CRUD complet des salles & équipements (Admin)	Livré	ManyToMany Room-Equipment
CRUD classes & utilisateurs (Admin)	Livré	Création, suppression, reset MDP
Réservation pour soi-même ou pour autrui	Livré	Admin : tous, Coord : ses étudiants
Détection des conflits de réservation	Livré	RoomRepository::isAvailable()
Annulation (soft delete) des réservations	Livré	Historique conservé en BDD
Filtre de disponibilité par date et créneau	Livré	Toutes interfaces
Espace coordinateur avec gestion de classes	Livré	complets
Espace étudiant autonome	Livré	complets

Fonctionnalité	Statut	Remarque
Interfaces CSS distinctes par rôle	Livré	Vert coord, bleu étudiant, rouge admin
Déploiement Docker	Livré	docker-compose fonctionnel

Point fort : L'intégralité des fonctionnalités prévues au cahier des charges a été livrée dans les 5 jours impartis, ce qui constitue le principal point fort de cette réalisation.

3. Points forts

3.1 Architecture technique solide

Le choix de Symfony 7 avec Doctrine ORM a permis une séparation claire des responsabilités grâce au pattern MVC. La gestion des rôles via le système de sécurité natif de Symfony (firewalls, access_control, IsGranted) garantit une protection robuste sans code redondant.

Point fort : Le pattern d'héritage de table (OneToOne User → Administrator / Coordinator / Student) permet de déterminer le rôle dynamiquement sans colonne 'roles' en BDD — solution élégante et performante.

3.2 Soft delete des réservations

L'annulation des réservations ne supprime pas les données physiquement — elle change le statut à 'canceled'. Cette approche préserve l'historique complet pour un suivi d'occupation futur et respecte la contrainte fonctionnelle de journalisation demandée dans le cahier des charges.

3.3 Interfaces adaptées à chaque rôle

Chaque profil dispose d'une interface CSS distincte (rouge pour l'admin, vert pour le coordinateur, bleu pour l'étudiant) avec un dashboard personnalisé. Cette différenciation visuelle améliore significativement l'expérience utilisateur et réduit les erreurs de navigation.

3.4 Sécurité des formulaires

Tous les formulaires de suppression et d'annulation sont protégés par des tokens CSRF générés par Symfony. Les vérifications de propriété des ressources sont systématiquement effectuées côté serveur (un coordinateur ne peut accéder qu'à ses propres classes, un étudiant ne peut annuler que ses propres réservations).

3.5 Conteneurisation Docker

L'environnement de développement est entièrement conteneurisé via docker-compose, ce qui garantit la reproductibilité de l'installation et simplifie le déploiement sur la VM Linux de l'école.

4. Limites actuelles

Malgré la complétude fonctionnelle, plusieurs limites ont été identifiées lors du développement ou sont inhérentes au délai d'une semaine.

4.1 Absence de notifications

L'application ne dispose d'aucun système de notification. Les utilisateurs ne sont pas informés lorsque :

- Une réservation est créée en leur nom par un coordinateur
- Une réservation est annulée par un administrateur
- Un créneau qu'ils souhaitaient réserver vient de se libérer
- Leur compte est créé ou leur mot de passe réinitialisé

> ⚠ **Limite :** L'absence de notifications force les utilisateurs à consulter régulièrement l'application pour rester informés, ce qui nuit à l'autonomie promise.

4.2 Pas de vue calendrier

La visualisation des réservations est uniquement tabulaire (listes). Il n'existe pas de vue calendrier hebdomadaire ou mensuelle permettant de visualiser d'un coup d'œil l'occupation des salles. Cette limite rend la planification moins intuitive pour les coordinateurs.

4.3 Absence de pagination

Les listes de salles, d'étudiants, de classes et de réservations ne sont pas paginées. Avec un nombre important de données, les pages pourraient devenir longues et difficiles à parcourir.

4.4 Pas d'authentification SSO

L'intégration avec le compte O365 de MediaSchool IRIS Nice est explicitement hors périmètre du projet. Les utilisateurs doivent donc gérer un compte supplémentaire distinct de leur compte école.

4.5 Réinitialisation du mot de passe manuelle

La réinitialisation du mot de passe nécessite l'intervention d'un administrateur. Il n'existe pas de fonctionnalité "mot de passe oublié" envoyant un lien de réinitialisation par email automatiquement.

4.6 Absence de récurrence dans les réservations

Il est impossible de créer une réservation récurrente (ex : tous les lundis de 9h à 11h pendant 8 semaines). Chaque créneau doit être réservé manuellement, ce qui est contraignant pour les intervenants ayant des cours réguliers.

5. Risques identifiés

Risque	Probabilité	Impact	Mesure corrective
Croissance du nombre d'utilisateurs (pas de pagination)	Élevée	🟠 Moyen	Implémenter la pagination KnpPaginatorBundle
Conflit de réservation concurrent (race condition)	Faible	🔴 Élevé	Ajouter une contrainte UNIQUE en BDD sur (room_id, start, end) ou utiliser des transactions
Perte de données BDD (pas de sauvegarde automatique)	Moyenne	🔴 Élevé	Configurer une sauvegarde automatique quotidienne du volume Docker
Accès non autorisé via manipulation d'URL	Faible	🔴 Élevé	Les vérifications côté serveur sont en place — à maintenir sur toute nouvelle route
Mots de passe faibles des utilisateurs (corrigé)	Élevée	🟠 Moyen	Renforcer la contrainte de validation (longueur, complexité) dans RegistrationFormType
Indisponibilité du serveur Docker	Faible	🟠 Moyen	Documenter la procédure de redémarrage et prévoir un environnement de backup

6. Pistes d'amélioration futures

6.1 Court terme (priorité haute)

Amélioration	Description
Pagination des listes	Intégrer KnpPaginatorBundle sur les listes salles, étudiants, réservations
Mot de passe oublié	Envoyer un lien de réinitialisation par email (Symfony Mailer + Mailtrap)

6.2 Moyen terme

Amélioration	Description
Vue calendrier	Intégrer FullCalendar.js pour visualiser les réservations en vue semaine/mois
Notifications email	Envoyer un email automatique à chaque création/annulation de réservation
Réservations récurrentes	Permettre la création d'une série de créneaux (ex : chaque lundi pendant N semaines)
Export PDF du planning	Générer un PDF du planning hebdomadaire d'une salle ou d'un utilisateur
Sauvegarde automatique BDD	Configurer un cron docker pour dumper la BDD quotidiennement

6.3 Long terme

Amélioration	Description
Authentification SSO O365	Intégrer l'authentification via le compte Microsoft O365 de l'école (OAuth2)
Application mobile	Développer une app mobile (React Native ou PWA) pour réserver depuis smartphone
Tableau de bord analytique	Statistiques d'occupation des salles : taux d'utilisation, heures de pointe, salles les plus réservées
API REST publique	Exposer une API REST documentée (Swagger/OpenAPI) pour intégration avec d'autres outils de l'école

Amélioration	Description
Gestion des équipements mobiles	Gérer les équipements prêtables (vidéoprojecteurs mobiles, câbles HDMI...) distinct des équipements fixes

7. Retour d'expérience personnel

7.1 Compétences développées

- Maîtrise approfondie de Symfony 7 : sécurité, formulaires, Doctrine ORM, repositories personnalisés
- Gestion d'un projet full-stack en autonomie complète sur une semaine
- Conception de modèle de données relationnelle (MCD → MLD → MPD)
- Mise en place d'un environnement Docker pour le développement et le déploiement
- Production d'un corpus documentaire professionnel complet (CDC, guides, modèle de données, bilan)

7.2 Difficultés rencontrées

Difficulté	Solution apportée
Pré-sélection de salle dans le formulaire de réservation	Utiliser form.get('room').setData() avant handleRequest() au lieu de l'option 'data'
Gestion des rôles sans colonne 'roles' en BDD	Calcul dynamique dans getRoles() selon la présence d'un profil lié (administrator/coordinator)
Token CSRF invalide dans les partials Twig	Harmoniser les noms de token entre le partial et le controller (remove_coordinator_ vs remove_student_)
Migration ManyToMany Room-Equipment	Vider les données en conflit avant de rejouer la migration après ajout de la contrainte UNIQUE
Route /registration bloquée pour les non-connectés	Ajouter PUBLIC_ACCESS dans security.yaml avant la règle générale ROLE_USER
Syntaxe Twig avec guillemets dans onsubmit	Utiliser des guillemets doubles dans le filtre date() à l'intérieur d'un attribut HTML en guillemets simples

7.3 Estimation du temps passé

Phase	Temps estimé	Remarque
Analyse & conception (MCD/MLD/MPD)	1 jour	Réalisé en parallèle du développement
Mise en place Docker + Symfony + BDD	2 jours	Configuration initiale du projet
Développement espace Admin	7 jours	CRUD salles, classes, utilisateurs, réservations
Développement espace Coordinateur	5 jours	C1 à C10 — formulaires, sécurité, CSS
Développement espace Étudiant	3 jours	S1 à S6 — similaire au coordinateur
Tests & corrections de bugs	7 jours	Débogage CSRF, formulaires, migrations
Documentation (6 livrables)	5 jours	CDC, guides, modèle de données, bilan, README

8. Conclusion

Le projet RoomBooking a été mené à terme dans les délais impartis avec l'intégralité des fonctionnalités du cahier des charges livrées et fonctionnelles. L'architecture choisie (Symfony 7, Doctrine ORM, Docker) a permis un développement structuré et maintenable malgré la contrainte de temps d'une semaine.

Les principales limites identifiées (absence de notifications, de vue calendrier et de réservations récurrentes) sont des fonctionnalités d'amélioration de l'expérience utilisateur qui n'impactent pas le fonctionnement core de l'application. Elles constituent des pistes concrètes pour une version future.

Ce projet a permis de mettre en pratique l'ensemble des compétences du référentiel BTS SIO SLAM : conception de base de données, développement backend sécurisé, interfaces frontend responsive, gestion des droits utilisateurs et production documentaire professionnelle.

Point fort : RoomBooking répond pleinement à l'objectif de digitalisation de la gestion des salles de MediaSchool IRIS Nice, avec une base technique solide permettant des évolutions futures sans réécriture majeure.

MediaSchool IRIS Nice — BTS SIO SLAM — Avril 2026